8 Database fragmentation

Fragmentation is the task of dividing a table into a set of smaller tables. The subsets of the table are called fragments. Fragmentation can be of three types: horizontal, vertical, and hybrid (combination of horizontal and vertical). Horizontal fragmentation can further be classified into two techniques: primary horizontal fragmentation and derived horizontal fragmentation.

Fragmentation should be done in a way so that the original table can be reconstructed from the fragments. This is needed so that the original table can be reconstructed from the fragments whenever required. This requirement is called "reconstructiveness."

8.1 Objectives of Database Fragmentation

- *Easy usage of Data*: It makes most frequently accessed set of data near to the user. Hence these data can be accessed easily as and when required by them.
- *Efficiency* : It in turn increases the efficiency of the query by reducing the size of the table to smaller subset and making them available with less network access time.
- *Security* : It provides security to the data. That means only valid and useful records will be available to the actual user. The DB near to the user will not have any unwanted data in their DB. It will contain only the information, which is necessary for them.
- *Parallelism* : Fragmentation allows user to access the same table at the same time from different locations. Users at different locations will be accessing the same table in the DB at their location, seeing the data that are meant for them. If they are accessing the table at one location, then they have to wait for the locks to perform their transactions.
- *Reliability* : It increases the reliability of fetching the data. If the users are located at different locations accessing the single DB, then there will be huge network load. This will not guarantee that correct records are fetched and returned to the user. Accessing the fragment of data in the nearest DB will reduce the risk of data loss and correctness of data.
- *Balanced Storage* : Data will be distributed evenly among the databases in DDB.

8.2 Issues in Fragmentation

- *Completeness* : While creating the fragment, partial records in the table should not be considered. Fragmentation should be performed on whole table's data to get the correct result. For example, if we are creating fragment on EMPLOYEE table, then we need to consider whole EMPLOYEE table for constructing fragments. It should not be created on the subset of EMPLOYEE records.
- *Reconstructions* : When all the fragments are combined, it should give whole table's data. That means whole table should be able to reconstruct using all fragments. For example all fragments' of EMPLOYEE table in the DB, when combined should give complete EMPLOYEE table records.

- *Disjointedness* : There should not be any overlapping data in the fragments. If so, it will be difficult to maintain the consistency of the data. Effort needs to be put to create same replication in all the copies of data. Suppose we have fragments on EMPLOYEE table based on location then, there should not be any two fragments having the details of same employee.

8.3 Types of Fragmentation

8.3.1 Vertical Fragmentation

In vertical fragmentation, the fields or columns of a table are grouped into fragments. In order to maintain reconstructiveness, each fragment should contain the primary key field(s) of the table. Vertical fragmentation can be used to enforce privacy of data.

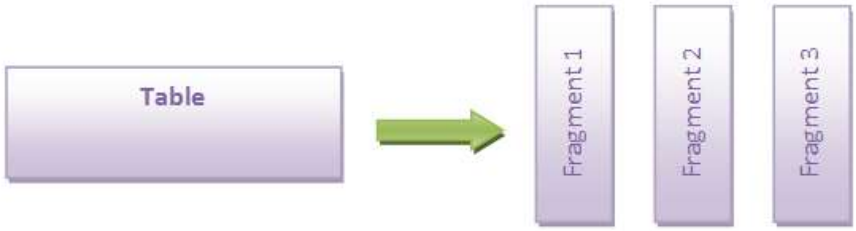
For example, let us consider that a University database keeps records of all registered students in a Student table having the following schema.

STUDENT

Regd_No	*Name*	*Course*	*Address*	*Semester*	*Fees*	*Marks*

Now, the fees details are maintained in the accounts section. In this case, the designer will fragment the database as follows −

```
CREATE TABLE STD_FEES AS
   SELECT Regd_No, Fees
   FROM STUDENT;
```

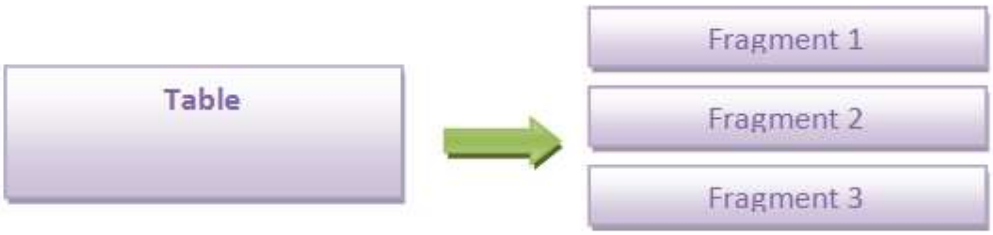


8.3.2 Horizontal Fragmentation

Horizontal fragmentation groups the tuples of a table in accordance to values of one or more fields. Horizontal fragmentation should also confirm to the rule of reconstructiveness. Each horizontal fragment must have all columns of the original base table.

For example, in the student schema, if the details of all students of Computer Science Course needs to be maintained at the School of Computer Science, then the designer will horizontally fragment the database as follows −

```
CREATE COMP_STD AS
   SELECT * FROM STUDENT
   WHERE COURSE = "Computer Science";
```

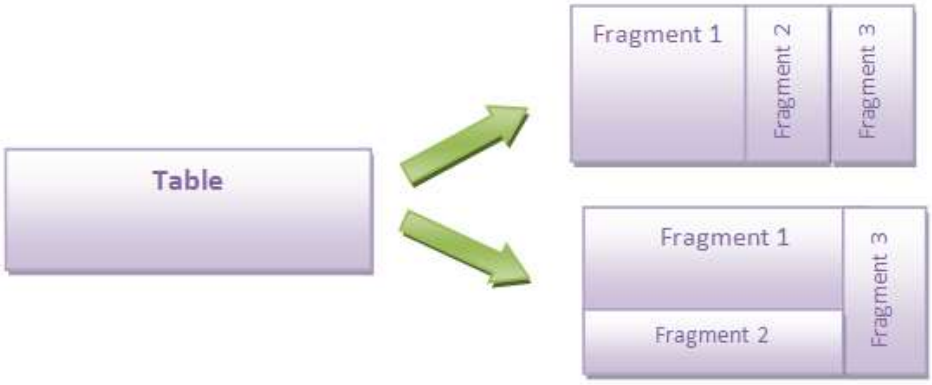


8.3.3 Hybrid Fragmentation

In hybrid fragmentation, a combination of horizontal and vertical fragmentation techniques are used. This is the most flexible fragmentation technique since it generates fragments with minimal extraneous information. However, reconstruction of the original table is often an expensive task.

Hybrid fragmentation can be done in two alternative ways −

- At first, generate a set of horizontal fragments; then generate vertical fragments from one or more of the horizontal fragments.
- At first, generate a set of vertical fragments; then generate horizontal fragments from one or more of the vertical fragments.



9 Database Replication

Database replication is the process in which the data is copied at multiple locations (Different computers or servers) to improve the *availability* of data.

9.1 Goals of data replication

The main objectives of data replication are:

1. Increase the availability of data.
2. Speed up the query evaluation.

9.2 Types of data replication

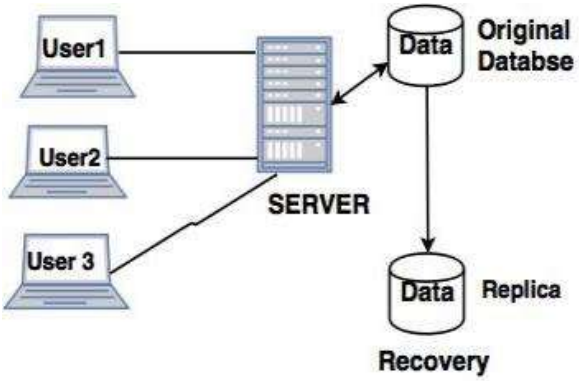
1. Synchronous Replication:
In synchronous replication, the replica will be modified immediately after some changes are made in the relation table. Thus, there is no difference between original data and replica.

2. Asynchronous replication:
In asynchronous replication, the replica will be modified after commit is fired on to the database.

9.3 Replication Schemes

9.3.1 Full Replication

In full replication scheme, the database is available to almost every location or user in communication network.



Full Replication Process In Distributed System

Advantages of full replication

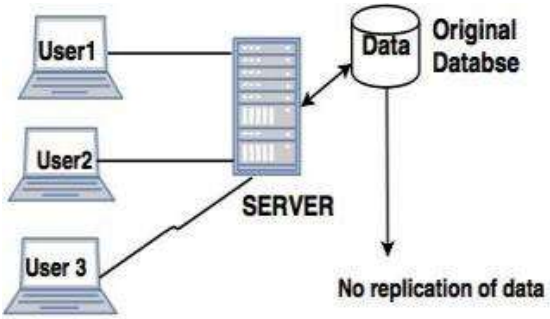
- High availability of data, as database is available to almost every location.
- Faster execution of queries.

Disadvantages of full replication

- Concurrency control is difficult to achieve in full replication.
- Update operation is slower.

9.3.2 No Replication

No replication means, each fragment is stored exactly at one location.



No Replication Process in Distributed Databases

Advantages of no replication

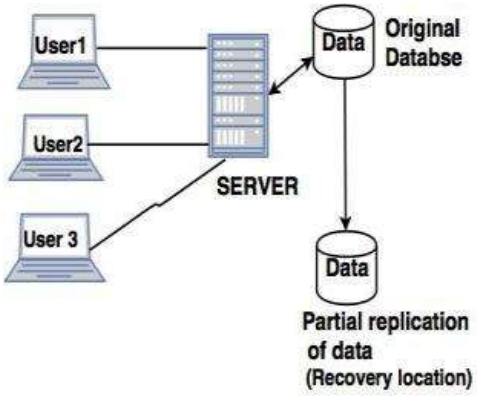
- Concurrency can be minimized.
- Easy recovery of data.

Disadvantages of no replication

- Poor availability of data.
- Slows down the query execution process, as multiple clients are accessing the same server.

9.3.3 Partial replication

Partial replication means only some fragments are replicated from the database.



Partial Replication Process In Distributed System

Advantages of partial replication

The number of replicas created for fragments depend upon the importance of data in that fragment.